Architecture Diagram and Module Description

Project: E-Commerce & Order Management System (Olist) · **Group:** TarushTanishkTanishq
Version: 1.0 (mid-semester) · Source: `docs/Design.md` §1–2

1. Architectural style

A **three-tier, layered client–server architecture**: a browser front end, a Flask REST API, and PostgreSQL 16. Business rules and access control live **inside the database tier** (constraints, triggers, stored procedures, roles and row-level security), so they hold for every client: the web app, `psql` and the ML script.

2. System architecture

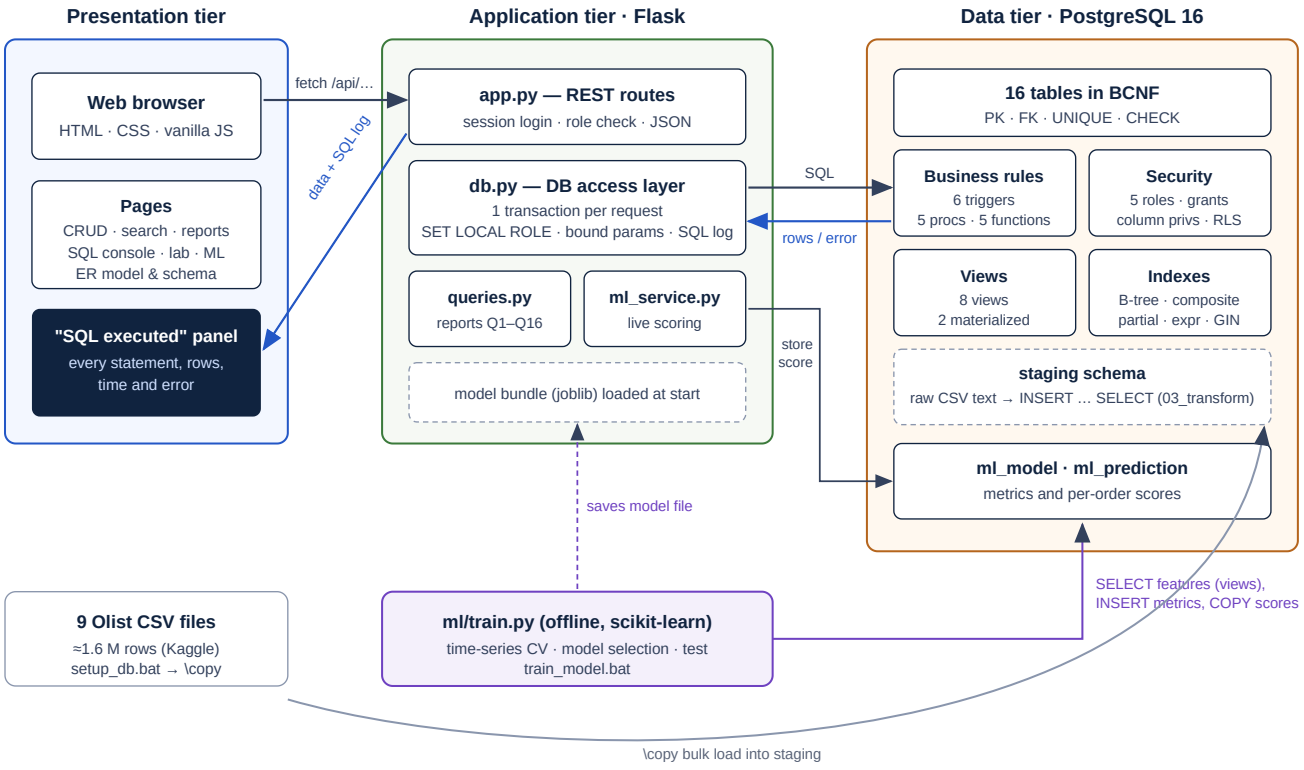
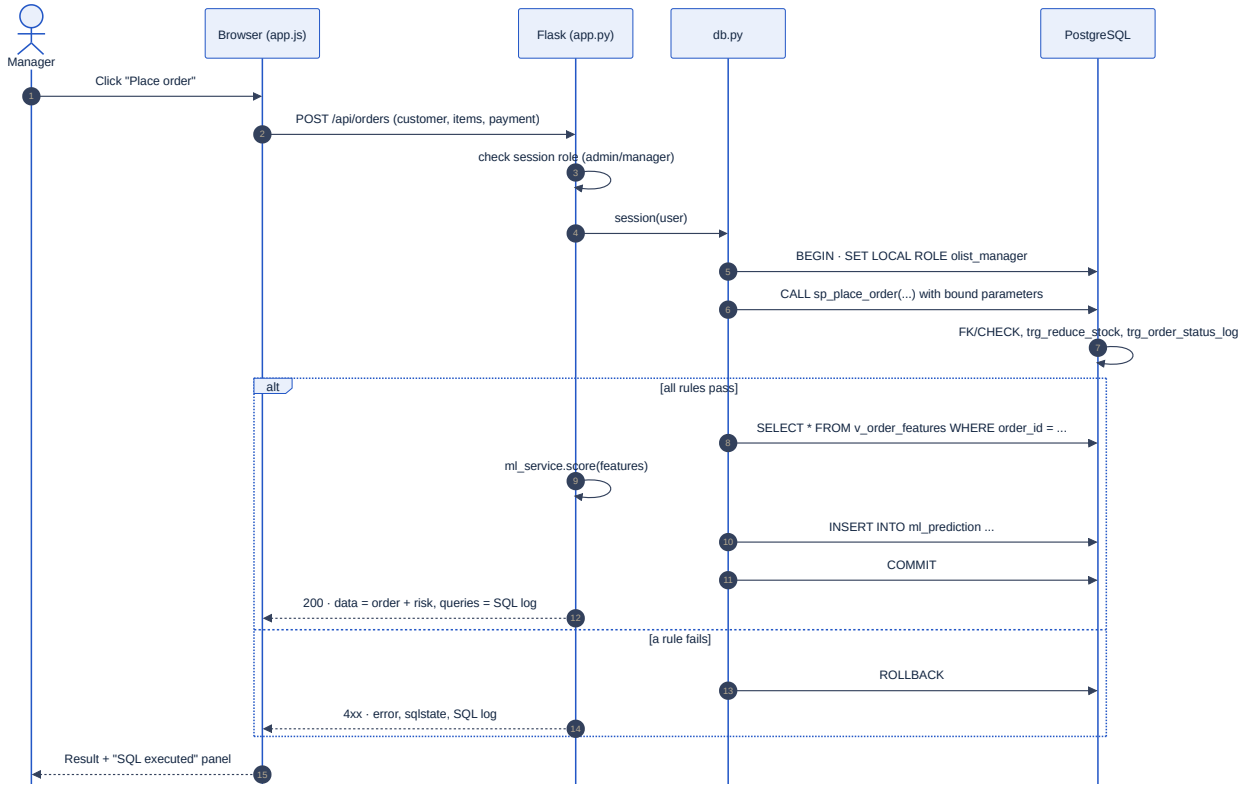


Figure 1: Three-tier architecture. Black arrows are requests (each runs as BEGIN · SET LOCAL ROLE · SQL · COMMIT/ROLLBACK), blue arrows are results; purple is the offline ML pipeline.

3. Request flow: Front end → SQL → Database → Result

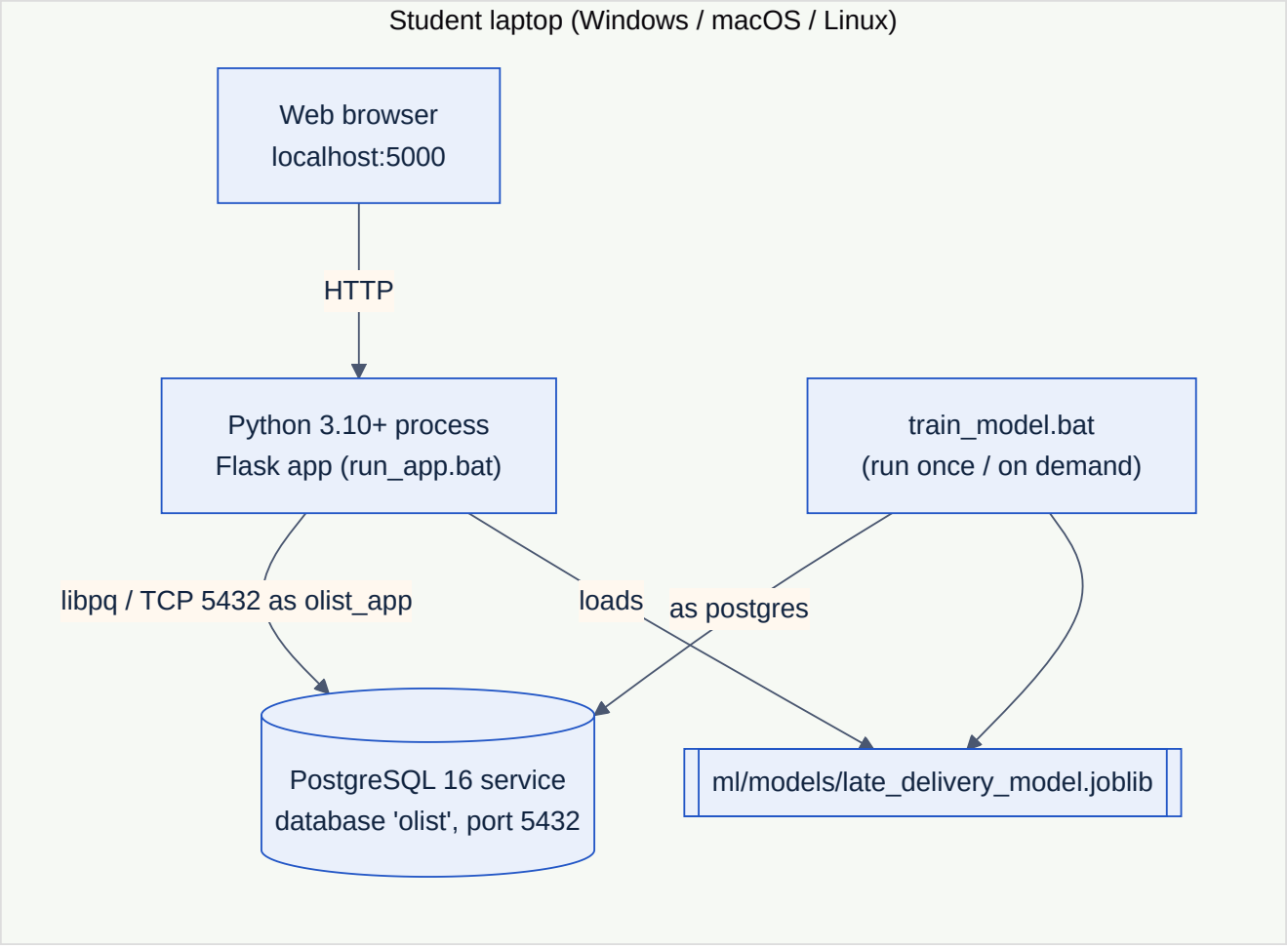


4. Module description

Module	Files	Responsibility	Owner
Schema	`db/01_schema.sql`	16 tables with PK, FK (`ON DELETE CASCADE` for owned rows, `RESTRICT` for referenced master data), UNIQUE and CHECK constraints	Tarush
Data pipeline	`db/02_staging_load.sql` , `db/03_transform.sql`	Load 9 CSVs (~1.6 M rows) as text into `staging` , clean and cast into normalized tables	Tarush
Indexes and views	`db/04_indexes.sql` , `db/05_views.sql`	B-tree, composite, partial, expression and GIN indexes; 8 views and 2 materialized feature views	Tarush
Business rules	`db/06_triggers.sql` , `db/07_procedures.sql`	Status state machine, audit log, delete guard, review rule, stock control, prediction invalidation; `sp_place_order` , `sp_update_status` , `sp_cancel_order` , `fn_login`	Tanishk
Security	`db/08_security.sql`	Group roles, least-privilege `olist_app` login, grants, column privileges, row-level security, demo users	Tanishk
SQL demos	`db/09 – 14`	Report queries, ACID, concurrency, DDL/DML, index and normalization demos	Tanishk, Tarush
DB access layer	`backend/db.py`	One transaction per request, `SET LOCAL ROLE` , SQL log (statement, rows, time, error)	Tanishk
REST API	`backend/app.py` , `backend/queries.py`	Login, CRUD for 6 entities, faceted search, reports + EXPLAIN + CSV, SQL console, schema, ML, transactions lab	Tanishk
ML service	`backend/ml_service.py`	Load the trained model bundle and score new orders live	Tanishq

Module	Files	Responsibility	Owner
ML training	`ml/common.py`, `ml/train.py`	Feature loading from SQL, time-series CV, model selection, threshold tuning, test evaluation, write results to the DB	Tanishq
Front end	`frontend/`	Single-page app: CRUD pages, search, reports, SQL console, transactions lab, ML page, ER & schema page, SQL panel	Tanishq
Tests	`db/15_test_suite.sql`, `tests/api_tests.py`	Automated database and end-to-end test cases	Tanishq (test lead), all

5. Deployment view



6. Key design decisions

Decision	Alternatives considered	Reason
Layered client / API / DB	Server-rendered monolith	Each member owns one layer and can test it on its own
Rules in the DBMS	Rules only in Python	Hold for every client; the DBMS is the course focus
Raw SQL via psycopg, no ORM	SQLAlchemy	The SQL executed must be visible to the user

Decision	Alternatives considered	Reason
Client-side parameter binding (`ClientCursor`)	Server-side binding	The SQL shown is byte-for-byte what PostgreSQL ran, and values are still escaped safely
Per-request `SET LOCAL ROLE`	One powerful DB user	PostgreSQL enforces least privilege and RLS itself
Vanilla JS, no build step	React / Angular	Zero setup for evaluators